MY-SQL NOTES

Data Table: Data in Database is always represented In the From of Data Table Having Collection of Rows and Columns

Database: Collection of Tables Belonging to One Organization or Computer Application Where Every Table Represents Some Unique Information.

Popular DBMS Products Were: Dbase, Clipper, Paradox, Foxbase, Foxpro and MS Access

Popular RDBMS Products are: MySQL, MySQL, SQLSERVER, Post Grace Firebase etc.

RDBMS (Relational Database Management System)

RDBMS: A set of related data tables in which one table is related to another by using a common column to establish relationship between parent table child table and data stored is valid according to the integrity constraints is called a RDBMS.

Integrity Constraints- A set of rules.

Integrity Constraints: A set of rules and any RDBMS including MYSQL has three basic rules.

Domain-Column; Tuple-Row

(i) Domain Integrity: It means according to this rule the data stored under every particular domain (or) column must reflect proper information compared to its headings (or) column name. This rule is implemented practically in any RDBMS by using different data types.

(ii) Entity Integrity: According to this constraint, every row stored under a table must be uniquely identifiable. It means there should be at least one column in a table, which will not have repetitive value (or) Null Value. Practically, it is implemented in any RDBMS by using "Primary key".

(iii) Referential Integrity: When values of one column in a table are dependent on values of another column in another table having common data type.

Self-Referential Integrity: When values of one column in a table are dependent on values of another column within the same table, then it is called as "Self Referential Integrity". Practically, it is implemented by using Reference.

SQL: Structured Query Language.

It is a common language used by any RDBMS to do basic operations on their RDBMS. Following are the features of SQL: -

1. It is simple English like language and uses commands such as Select, Create, Drop etc., in its commands.

2. It is not having If, Loop, Variables and most of the commands are Single line commands.

3. To implement application Logics, SQL has got extension Language popularly called as PL/SQL (Procedural Language of SQL).

4. The entire SQL has been divided into 4 major categories. They are-
    - Data Manipulation Language (SELECT, INSERT, UPDATE, DELETE)
    - Data Definition Language (CREATE, ALTER, DROP, TRUNCATE)
    - Transaction Control Language (COMMIT, ROLLBACK)
    - Data Control Language (GRANT, REOVE)

COMMIT=SAVE; ROLLBACK=UNDO

Note: Every SQL Command Must End With Semicolon(;) when we Are Using Command Line Interface(CLI)

Database Management System Vs. Relational Database Management System

Most of the DBMS were developed for supporting DOS; whereas most of the RDBMS work on Windows/Unix/Linux Systems/Operating Systems.

To See List of Databases We Use Command

SHOW DATABASES;

To see the list of the Tables, present, the following command is used: -

SHOW FULL TABLES;

For clearing the screen:

`Ctrl + L Or system cls`

CREATE: This command is used to create any database object including Tables, Views, and Sequence etc.,

Following is an example of creating a Table called STUDENT with the help of 4 columns (STNO, STNAME, DOA, FEES)

```
CREATE TABLE STUDENT (
   STNO INT (4),
   STNAME VARCHAR (30),
   DOA DATE,
   FEES DECIMAL (6, 0)
);
```

The important data types of SQL are:-

CHAR, VARCHAR2, INT/INTEGER, DECIMAL,

DATE: Stores only the date (`YYYY-MM-DD`).

DATETIME: Stores both date and time (`YYYY-MM-DD HH:MM:SS`).

VARCHAR2 is used to save Bytes according to the Characters of the Name.
To see the columns, present in a Table, DESCRIBE command is used

To See the Structure of Table Use

```
DESCRIBE STUDENT;
-- OR --
DESC DESCRIPT;
```

INSERT: In MySQL to Add or New Records, INSERT command is used.
There are four different types of INSERTS in MySQL, they are: -

i. INSERT ALL FIELDS ONE RECORD
ii. INSERT FEW FIELDS ONE RECORD
iii. INSERT ALL FIELDS MANY RECORD
iv. INSERT FEW FIELDS MANY RECORD

Note: in MySQL for CHARACTER, DATE field, information is inserted within the Single quotes. Where as Numerical fields are entered directly

1)INSERT ALL FIELDS ONE RECORD

INSERT INTO STUDENT VALUES (1, 'KRISHNA', '2006-07-07', 3000);

2)INSERT FEW FIELD ONE RECORD

INSERT INTO STUDENT (STNO, STNAME) VALUES (2, 'RAVI');

3)INSERT ALL FIELDS MANY RECORDS

INSERT INTO STUDENT (STNO, STNAME, DOA, FEES) VALUES (3, 'AMIT', '2024-01-15', 5000), (4, 'SNEHA', '2024-02-10', 5500), (5, 'PRIYA', '2024-03-05', 6000);

4)INSERT FEW FIELDS MANY RECORDS

INSERT INTO STUDENT (STNO, STNAME) VALUES (6, 'AMIT'), (7, 'SNEHA'), (8, 'PRIYA');

Note: MySQL has feature called the edit command (or the shortcut \e). Here is how you use it:

Type \e and press Enter.

Your system's default text editor (like Notepad on Windows or Vim/Nano on Linux) will open.

Write your SQL query without a semicolon (just like in Oracle).

Save the file and Close the editor.

SELECT: To see the Records present in a Table, SELECT command is used as show below: -

SELECT * FROM STUDENT;

UPDATE: To Change the existing record present in a Table, we use UPDATE command with the SET option.

SET SQL_SAFE_UPDATES = 0;

UPDATE STUDENT SET FEES=11000 WHERE STNO=5;

UPDATE STUDENT SET STNO=6, FEES=15000 WHERE STNAME =’ RAVI’;

SET SQL_SAFE_UPDATES = 1;

DELETE: When we want to delete ONE, FEW, ALL THE RECORDS present in a Table, following respective commands are used: -

DELETE FROM STUDENT WHERE STNO=4;

DELETE FROM STUDENT WHERE FEES>=10000;

DELETING ALL THE RECORDS:

DELETE FROM STUDENT;

To remove the Table itself, DROP Command is used: -

DROP TABLE STUDENT;

PROJECTION: When we want to see all the rows, but only few columns of a table, then instead of using *, we use the column names in the SELECT statement, which is called as Projection.

SELECT EMPNO, ENAME, JOB, SAL FROM EMP;

RESTRICTION: When we want to filter the rows of a Table in the query result by giving one or more condition with the help of WHERE clause, then it is called as RESTRICTION.

i. **QUERY WITH SINGLE CONDITION**

SELECT * FROM EMP WHERE SAL>=3000;

SELECT * FROM EMP WHERE JOB='MANAGER';

ii. **QUERIES BASED ON TWO CONDITIONS**

SELECT * FROM EMP WHERE JOB='CLERK' AND SAL>1200;

iii. **QUERY BASED ON THREE CONDITIONS**

SELECT * FROM EMP WHERE DEPTNO=10 AND JOB='SALESMAN' AND SAL>=1000;

SELECT * FROM EMP WHERE (DEPTNO=10 OR DEPTNO=20) AND JOB='SALESMAN';

SELECT * FROM EMP WHERE JOB! ='CLERK'

SELECT * FROM EMP WHERE NOT JOB='CLERK';

COLUMN ALIAS: Whenever query results are displayed, they will have the some heading as their column alias (or) the Expression given by you, which sometime may not look proper. In such case, column alias can be used, which has to be a single (or) it may contain an underscore, but it cannot be split in words.

SELECT ENAME, SAL*12 AS ANNUAL_SAL FROM EMP;

CONCATINATION: Whenever query results are displayed, there will be some gap between one column to another, which can be removed by using double pipe symb0ols called CONCATINATION.

SELECT CONCAT (ENAME, JOB) FROM EMP;

LITERLS: When we want to replace certain information in every row of the result, but it is neither a column name, nor a column alias, then it is called as LITERAL.

SELECT CONCAT (ENAME, ' WORKS AS ', JOB, ' IN DEPARTMENT ', DEPTNO) AS EMPLOYEE_DETAILS FROM EMP;

DISTINCT: When we want to remove the repetitive rows from the query result, we use DISTINCT.

SELECT DISTINCT JOB FROM EMP;

NULL VALUE HANDLING: In MySQL, whenever a value gets added to Null, it also becomes Null. To overcome this problem, we use IFNULL function as shown below: -

SELECT ENAME, SAL, COMM, SAL + IFNULL (COMM, 0) AS TOTAL_INCOME FROM EMP;

ASCENDING/ DESCENDING ORDER: When we want to display the query results in the Ascending Order; we used ORDER BY, in the Descending order by 'DSESC'.

Note: The option order by should always be at the end of the SQL statement. Order by can be implemented on One or More columns at the same time.

SELECT * FROM EMP ORDER BY ENAME;

SELECT * FROM EMP ORDER BY SAL DESC;

SELECT DEPTNO, ENAME FROM EMP ORDER BY DEPTNO, ENAME;

SELECT * FROM EMP WHERE HIREDATE >= STR_TO_DATE ('01/01/1981', '%d/%m/%Y') AND HIREDATE <= STR_TO_DATE ('31/12/1981', '%d/%m/%Y');

LIMIT(TO RESTRICT ROWS TO BE DISPLAYED)

The `LIMIT` clause accepts one or two non-negative integer arguments:

- **`LIMIT row_count`**: Returns the specified maximum number of rows from the beginning of the result set.
- **`LIMIT offset, row_count`**: Specifies both a starting point (offset) and the maximum number of rows to return. The offset of the first row is `0`, not `1`

SELECT EMPNO,ENAME,SAL FROM EMP LIMIT 3;

SELECT EMPNO,ENAME,SAL FROM EMP LIMIT 3,3;

SQL OPERATORS: Following four options are called as SQL operators. They are:-

i. IN
ii. BETWEEN AND
iii. LIKE
iv. IS NULL

IN: When we want to select a few rows randomly from a large Table instead of using too many or operators, we should use one IN operator with a series of values given the brackets.

SELECT * FROM EMP WHERE EMP IN (7766, 7788, 7900);

BETWEEN AND: When we want to select few records inclusive of values given in the multiple conditions, then use operator BETWEEN AND.

SELECT * FROM EMP WHERE SAL BETWEEN 3000 AND 5000;

LIKE: When we want to search few records based on Character field with Wild Characters like *, then LIKE operator is used. We Can Use Two Place Holders
% Symbol For N Number of Unknown Characters And Under Score (_) for One Unknown Character.

Name to be displayed starting with ‘A’

SELECT * FROM EMP WHERE ENAME LIKE ‘A%’;

Name to be displayed ending with ‘S’

SELECT * FROM EMP WHERE ENAME LIKE ‘%S’;

Name to be displayed Having with ‘A’

SELECT * FROM EMP WHERE ENAME LIKE ‘%A%’;

Name is exactly 5 letters long: `SELECT * FROM EMP WHERE ENAME LIKE '-----';`

Second letter is 'A': `SELECT * FROM EMP WHERE ENAME LIKE '_A%';`

Starts with 'S' and is at least 3 letters long: `SELECT * FROM EMP WHERE ENAME LIKE 'S__%';`

IS NULL: Using this operator, we can fetch those rows where information is ‘Blank’

SELECT * FROM EMP WHERE COMM IS NULL;

DATA EXTRACTION FROM MORE THAN ONE TABLE:

JOINS: In SQL, a **Join** is a clause used to combine rows from two or more tables based on a related column between them. While a `SELECT` statement usually retrieves data from a single table, Joins allow you to link tables together to create a comprehensive view of your data.

EQUI JOIN (NATURAL JOIN)
NON EQUI JOIN (INNER JOIN)
OUTER JOIN (LEFT RIGHT FULL OUTER JOIN)
SELF JOIN
CROSS JOIN / CARTESIAN JOIN

(i). EQUI JOIN: This join can be used only when there is at least One Common column between the Tables from where data is being extracted.

SELECT EMPNO, ENAME, DNAME, LOC FROM EMP, DEPT WHERE EMP.DEPTNO=DEPT.DEPTNO;

SELECT EMPNO, ENAME, EMP.DEPTNO, DNAME, LOC FROM EMP, DEPT WHERE EMP.DEPTNO=DEPT.DEPTNO;

SELECT EMPNO, ENAME, EMP.DEPTNO, DNAME, LOC FROM EMP E, DEPT D WHERE E. DEPTNO=D.DEPTNO;

(ii). NON EQUI JOIN: When we want to fetch the data from more than one Table, but they don't have any common column between then, then we use NON EQUI JOIN with the help of operator NON EQUI JOIN.

SELECT EMPNO, ENAME, SAL, GRADE FROM EMP, SALGRADE WHERE SAL BETWEEN LOSAL AND HISAL;

Three Table Joining:
SELECT EMPNO, ENAME, DNAME, LOC, SAL, GRADE FROM EMP, DEPT, SALGRADE WHERE EMP.DEPTNO=DEPT.DEPTNO AND SAL BETWEEN LOSAL AND HISAL;

(iii). **OUTER JOIN:** If we want to extract even non-matching rows in the result from both the Tables even though there is less information in one Table compared to another; then we use OUTER JOIN. There Are Three Outer Joins they are
LEFT OUTER JOIN, RIGHT OUTER JOIN, FULL OUTER JOIN

SELECT EMPNO, ENAME, D. DEPTNO, DNAME, LOC FROM EMP E RIGHT JOIN DEPT D ON E. DEPTNO = D. DEPTNO;

SELECT E. ENAME, E. DEPTNO, D. DNAME FROM EMP E LEFT JOIN DEPT D ON E. DEPTNO = D. DEPTNO;

Ex of Full Outer Join

SELECT E.ENAME, D.DEPTNO, D.DNAME FROM EMP E LEFT JOIN DEPT D ON E.DEPTNO = D.DEPTNO UNION SELECT E.ENAME, D.DEPTNO, D.DNAME FROM EMP E RIGHT JOIN DEPT D ON E.DEPTNO = D.DEPTNO;

(iv). SELF JOIN: When we want to join a Table back to the same Table again, then it is called as SELF JOIN.

In case of SELF JOIN, one Table name will be used twice in the SELECT statement by using different aliases & they will be joined by giving a condition.

SELECT E.EMPNO AS EMPLOYEE_NO, E.ENAME AS EMPLOYEE_NAME, M.EMPNO AS BOSS_NO, M.ENAME AS BOSS_NAME FROM EMP E, EMP M WHERE E.MGR = M.EMPNO;

Or

SELECT E.ENAME AS Employee, COALESCE(M.ENAME, 'THE BIG BOSS') AS Manager FROM EMP E LEFT JOIN EMP M ON E.MGR = M.EMPNO;

In SQL, the UNION operator is used to combine the result sets of two or more SELECT statements into a single result set The Statements may be on same table or different table but column names must be same in both the statements

UNION: Combines the results of two queries and removes duplicate rows. It performs an internal "sort" to ensure every row in the final list is unique.

UNION ALL: Combines the results of two queries and keeps all rows, including duplicates. Because it doesn't search for duplicates, it is significantly faster than UNION.

```
SELECT JOB FROM EMP WHERE DEPTNO=10
UNION
SELECT JOB FROM EMP WHERE DEPT NO=20;
```

```
SELECT JOB FROM EMP WHERE DEPTNO=10
UNION ALL
SELECT JOB FROM EMP WHERE DEPT NO=20;
```

INTERSECT: If we use this operator, it will bring only those rows, which are present in both the queries.

```
SELECT JOB FROM EMP WHERE DEPTNO=10
INTERSECT
SELECT JOB FROM EMP WHERE DEPTNO=20;
```

MINUS: This operator will give only those rows which are present in first query but not present in second query.

```
SELECT JOB FROM EMP WHERE DEPTNO=10
MINUS
SELECT JOB FROM EMP WHERE DEPTNO-20;

SELECT DEPTNO FROM DEPT
MINUS
SELECT DEPTNO FROM EMP
```

DUAL: It is a special system Table given by the MYSQL which acts like a White Board or Notepad through which any Dummy results can be displayed and scratched OFF.

```
SELECT NOW () FROM DUAL;
 -- The "MySQL style" (Cleaner/Standard) SELECT NOW ();
 -- Selecting a string or calculation SELECT 'Hello World' AS Greeting, 5 * 5 AS Result;
```

FUNCTIONS OF SQL:

In MySQL, there are 5 different types of Functions. They are:-

(i). CHARACTER FUNCTIONS
(ii). NUMERICAL FUNCTIONS
(iii). DATE FUNCTIONS
(iv). DATE FORMAT FUNCTIONS
(v). GROUP FUNCTIONS

Following 10 are different CHARACTER FUNCTIONS:

(i). LOWER: This command is used to convert Capital Letters to Small cases.
SELECT LOWER (ENAME) FROM EMP;
SELECT LOWER (‘RAILWAY’) FROM DUAL;

(ii). UPPER: This command is used to convert Small Letters to Capital cases.
SELECT UPPER (‘railway’) FROM DUAL;

(iii). **INITCAP:** This function converts the first Letter to Capital, rest to Small Letters.

```
SELECT CONCAT (UPPER (SUBSTRING (ENAME, 1, 1)), LOWER
(SUBSTRING (ENAME, 2))) AS ENAME FROM EMP;
```

(iv). **RPAD:** Whenever we want to fill up the right side blank spaces with special characters like * to avoid manipulation, we use RPAD.

```
SELECT RPAD (ENAME, 10, '*') FROM EMP;
```

(v). **LPAD:** This command is used to fill up the Left side blank spaces especially for Numerical fields.

```
SELECT LPAD(SAL, 7, '#') FROM EMP;
```

LENGTH: This command is used to get the length of the character length.

```
SELECT ENAME, LENGTH (ENAME) FROM EMP
```

(vi). **INSTR:** This command is called as In string Function. It is used to check whether a particular character is present in a string or not and if present, what is its position in the sting.

```
SELECT ENAME, INSTR (ENAME, 'A') FROM EMP;
```

(vii). **SUBSTR:** It is popularly called as Substring function is used to extract one or few characters from a given string. Function requires two numbers as parameters. The first number represents from where to start, the second number represents how many characters to be removed.

```
-- Standard MySQL version SELECT SUBSTRING ('RAILWAYS', 1, 4); -
- Oracle-style (also works in MySQL) SELECT SUBSTR ('RAILWAYS', 1, 4)
  FROM DUAL;
```

(viii). **REPLACE:** When we want to replace one string with another temporarily only in the result, then this function is used.

```
SELECT JOB, REPLACE (JOB, 'SALESMAN', 'MARKETING') FROM EMP;
```

(ix). **CASE:** As if conditions are not possible with the SQL statements, we use CASE instead of IF. In the function, we can give one General parameter, which acts like elseif condition.

```
SELECT
  ENAME,
```

```
   JOB,
   SAL,
   CASE JOB
      WHEN 'CLERK' THEN SAL * 10/100
      WHEN 'SALESMAN' THEN SAL * 20/100
      ELSE SAL * 30/100
   END AS HRA
FROM EMP;
```

```
SELECT ENAME, CASE
WHEN COMM IS NULL THEN 'COMMISSION IS NULL' WHEN COMM = 0 THEN
'COMMISSION IS ZERO' ELSE CAST (COMM AS CHAR)
END AS
COMM FROM EMP;
```

NUMERICAL FUNCTIONS: There are 8 types of Numerical functions. They are:-

(i). ROUND: It is used for rounding the Decimal part of the specific position. SELECT ROUND (200.2555) FROM DUAL; Result=200.25

(ii). CEIL: Irrespective of the Decimal Number if we want a Decimal number always rounded to the next nearest Integer, then we use Ceil.

SELECT CEIL (200.15) FROM DUAL; Result=200

(iii). FLOOR: It is opposite to the Ceil Function means; irrespective of the Decimal Numbers, it will always be rounded to the Current Integer.

SELECT FLOOR (200.95) FROM DUAL; Result=200

(iv). POWER: It is used to get the result of a square of a given number.

SELECT POWER (10, 3) FROM DUAL;

(v). SQRT: It is used to get the square root of the Number.

SELECT SQRT (25) FROM DUAL; Result =5

(vi). SIGN: This will give the result in terms of 1 or -1 representing whether the result achieved is Positive/Negative after arithmetical expression is evaluated.

SELECT COMM, SIGN (-COMM) FROM EMP;

(vii). **ABS:** This function called as Absolute function is used to convert Negative Numbers to Positive.

SELECT ABS (-500 *2) FROM DUAL; Result=1000

(viii). **MOD:** This function will give us the Reminder after one number is divided by another.
SELECT MOD (100,40) FROM DUAL; Result =20

DATE FORMAT FUNCTIONS: Following two options are called as Date Format Functions

SELECT DATE_FORMAT (NOW (), '%Y');
SELECT DATE_FORMAT (NOW (), '%W');
SELECT DATE_FORMAT (NOW (), '%H.%i.%s');
SELECT DATE_FORMAT (NOW (), '%W %D %M %Y');

How **STR_TO_DATE** Works

The function takes two arguments: `STR_TO_DATE (string, format)`. The format tells the database what each part of the string represents.

INSERT INTO EMP (EMPNO, ENAME, HIREDATE)
VALUES (1001, 'KRISHNA', STR_TO_DATE ('13/02/1998', '%d/%m/%Y'));

DATE FUNCTIONS: Following four options are called as Date functions. They are:

SELECT ENAME, TIMESTAMPDIFF (MONTH, HIREDATE, NOW ()) AS NO_OF_MONTHS FROM EMP;

-- Years between
(divide by 12 or just change the unit to YEAR)

SELECT ENAME, TIMESTAMPDIFF (YEAR, HIREDATE, NOW()) AS NO_OF_YEARS FROM EMP;

(i). DATE_ADD**:** Using this function, we can get a future date (or) previous by adding specific number of months positively/negatively.

SELECT DATE_ADD (NOW (), INTERVAL 6 MONTH)
; -- Subtracting 6 months (Past Date)

SELECT DATE_ADD (NOW (), INTERVAL -6 MONTH);

-- OR you can use the specific subtraction function: SELECT DATE_SUB (NOW (), INTERVAL 6 MONTH);

(ii). **LAST_DAY:** This function will give the last date of any month like 31, 30 even 28 in case of February.

SELECT LAST_DAY (NOW ());

GROUP FUNCTIONS: Following five options are called as group Functions because they look meaningful when implemented on an entire Table or atl3east a set of rows. They are: -

(i). **AVG:** This function gives the Average of a Numerical field.
SELECT AVG (SAL) FROM EMP;

(ii). **MIN:** This function gives the lowest value in a Numerical field.
SELECT MIN (SAL) FROM EMP WHERE DEPTNO=10 OR DEPTNO=20;

(iii). **MAX:** This function gives the highest value in a Numerical field.
SELECT MAX (SAL) FROM EMP WHERE JOB=('MANAGER');

(iv). **COUNT:** This function is used to count the Number of rows. It will not include Null values when counting * is used as parameter with the count function. However, if you fell that Null valves are present in the field instead of u sing *, you can use the field name also

SELECT COUNT (*) FROM EMP;
SELECT COUNT (COMM) FROM EMP;

SELECT DEPTNO, COUNT(*) AS HEADCOUNT, SUM(SAL) AS TOTAL_SPEND, AVG(SAL) AS AVERAGE_SALARY, MIN(SAL) AS LOWEST_SAL, MAX(SAL) AS HIGHEST_SAL FROM EMP GROUP BY DEPTNO ORDER BY TOTAL_SPEND DESC;

(v). **GROUP BY:** When we want to implement a Group function or more than One group at the same time, then GROUP BY clause should be used.

SELECT DEPTNO, MIN(SAL) FROM EMP GROUP BY DEPTNO;

SELECT JOB, MAX(SAL) FROM EMP GROUP BY JOB;

SELECT JOB, MAX(SAL) FROM EMP WHERE JOB=’PRESIDENT’ GROUP BY JOB; (Particular Column elimination)

(vi). **HAVING:** If where clause is used to filter the rows present in Table, HAVING clause is used to filter the rows arrived out of a Group Result.

SELECT JOB, MAX(SAL) FROM EMP GROUP BY JOB HAVING COUNT (JOB)>2;

SELECT DEPTNO, COUNT(DEPTNO) FROM EMP GROUP BY DEPTNO;

SELECT COUNT (EMPNO) FROM EMP GROUP BY EMPNO.HAVING COUNT (EMPNO)>1;

SUB QUERIES:

Whenever we want to extract the values of a Group function with single rows or when we want to extract the known values after extracting the unknown values then we use sub queries. When a Query Result is Dependent on Another Query Then it is a Subquery.

In case of sub queries, there will be minimum two queries, one called as Main or Outer query and other called as Inner Query.

For one main query, there can be a maximum of 255 inner queries. But usually, we use two or at the maximum three inner queries. In case of sub queries, always the inner most query gets executed first. Its result will be secretly passed to the outer query for further execution.

SELECT ENAME, JOB, SAL FROM EMP WHERE SAL=(SELECT MAX (SAL) FROM EMP);

SELECT ENAME, JOB FROM EMP WHERE ENAME != 'SCOTT' AND JOB = (SELECT JOB FROM EMP WHERE ENAME = 'SCOTT');

SELECT ENAME, JOB, MGR FROM EMP WHERE MGR = (SELECT EMPNO FROM EMP WHERE JOB = 'PRESIDENT');

SELECT MAX(SAL) FROM EMP WHERE SAL < (SELECT MAX(SAL) FROM EMP);

```
SELECT ENAME, SAL
FROM (
   SELECT ENAME, SAL,
```

```
        DENSE_RANK() OVER (ORDER BY SAL DESC) as salary_rank
    FROM EMP
) AS ranked_emp
WHERE salary_rank <= 3;
```

This Works Only In MySql

```
SELECT ENAME, SAL
FROM EMP
ORDER BY SAL DESC
LIMIT 3;
```

DIFFERENT TYPES OF QUERIES:

Display ENAME, SAL, DNAME, & JOB of all those who are getting salary more than the highest salary of any clerk.

SELECT ENAME, JOB, SAL, DNAME, LOC FROM EMP JOIN DEPT ON EMP.DEPTNO = DEPT.DEPTNO WHERE SAL > (SELECT MAX(SAL) FROM EMP WHERE JOB = 'CLERK');

Display ENAME, SAL, DEPTNO & their LOC, their Grade for all those who are getting salary more than any persons salary working in DEPTNO=20.

SELECT ENAME, JOB DEPTNO, LOC, GRADE, SAL FROM EMP, DEPT, SALGRADE WHERE EMP.DEPTNO=DEPT.DEPTNO AND SAL BETWEEN LOSAL AND HISAL AND SAL> (SELECT MAX (SAL) FROM EMP WHERE ENAME IN (‘ALLEN’, ADAMS, ‘JAMES’));

Create a query which will display ENAME, SAL DEPTNO & DNAME of those who are getting salary more than the highest salary of any person working in Research Department.

```
SELECT
    E.ENAME,
    E.JOB,
    E.DEPTNO,
    D.LOC,
    S.GRADE,
    E.SAL
FROM EMP E
JOIN DEPT D ON E.DEPTNO = D.DEPTNO
JOIN SALGRADE S ON E.SAL BETWEEN S.LOSAL AND S.HISAL
```

```
WHERE E.SAL > (SELECT MAX(SAL) FROM EMP WHERE ENAME IN ('ALLEN',
        'ADAMS', 'JAMES'));
```

Display the DEPTNO AND AVG SALARY, which is getting the highest average salary.

```
SELECT DEPTNO, AVG(SAL)
FROM EMP
GROUP BY DEPTNO
HAVING AVG(SAL) = (
   SELECT AVG(SAL)
   FROM EMP
   GROUP BY DEPTNO
   ORDER BY AVG(SAL) DESC
   LIMIT 1);
```

Display the ENAME, SALARY, COMM of those whose both SALARY and COMMISSION matches that of ALLEN.

```
SELECT ENAME, SAL, COMM
FROM EMP
WHERE (SAL, IFNULL(COMM, 0)) = (SELECT SAL, IFNULL(COMM, 0)
                        FROM EMP
                        WHERE ENAME = 'ALLEN')
AND ENAME != 'ALLEN';
```

CORRELATED QUERIES:

A correlative query works exactly opposite of a normal sub query means; In this case outer query will be executed first and its result will be passed to the inner query for further execution.

One more difference between a sub query and a correlated query is in case of sub query, the inner query will be executed only once; whereas in case of correlated query, the inner query will be executed so many times depending upon the candidate rows selected by the outer query.

Following are the steps for the way a correlated query gets executed: -

- **i.** Select all the candidate rows by using the outer query;
- **ii.** Send One by one candidate row to the inner query;
- **iii.** Execute the inner query by using one of the key columns present in the outer query;
- **iv.** Compare the result brought by the inner query with the candidate rows values to select or discard the candidate row.
- **v.** All the above steps will be executed till all the candidate rows are over.

Note: It is very difficult to distinguish between a sub query and a correlated query, but one method to identify could be by looking at the **Table alias** in the inner queries where clause;

```
SELECT E.ENAME, E.SAL, E.DEPTNO
FROM EMP E
WHERE E.SAL > (SELECT AVG(I.SAL)
        FROM EMP I
        WHERE I.DEPTNO = E.DEPTNO);
```

Following query will display the DEPTNO, DNAME, & LOC of those DEPTNO, which are present in DEPT TABLE but not present in EMP TABLE.

```
SELECT DEPTNO, DNAME, LOC FROM DEPT D WHERE NOT EXISTS (SELECT
* FROM EMP WHERE DEPTNO=D.DEPTNO);
```

TRANSACTION CONTROL LANGUAGE: COMMIT and ROLLBACK And SAVEPOINT are the two important sections of TCL.

COMMIT=SAVE; ROLLBACK UNDO

ALL THE INSERT, UPDATE & DELETE OPERATIONS (DML), WHICH WE DO DURING A SESSION ARE TEMPORARY ONLY IN THE MEMORY TILL WE GIVE COMMIT. EVEN WHEN THE SESSION IS SHUT DOWN PROPERLY OR THE DATABASE IS SHUTDOWN PROPERLY BY THE ADMINISTRATOR ALL THE DML OPERATIONS DONE BY US AUTOMATICALY GET COMMITTED.

But still **COMMIT** should be done as early as possible for the following reasons:-

a) To avoid any technical/power failure this could be lead to Loss of Data.
b) To make sure that every one present on the network will see the latest & correct data.
c) To avoid unwanted rollbacks.
d) To stop unknowingly troubling on the network.

ROLL BACK: It should be implemented before COMMIT. Once COMMIT is issued and transactions are made permanent on the hard disk, you cannot ROLLBACK back.

SAVEPOINT

This creates a "bookmark" inside a transaction. It allows you to roll back part of a transaction instead of the whole thing.

SAVEPOINT savepoint_name

```
UPDATE accounts SET balance = balance - 500 WHERE account_id = 101;
SAVEPOINT debit_done; -- Bookmark after first step

UPDATE accounts SET balance = balance + 500 WHERE account_id = 102;

-- If everything is correct:
COMMIT;

-- IF something went wrong after the first step:
-- ROLLBACK TO debit_done;
```

DATA DEFINITION LANGUAGE COMMANDS:

CREATE, ALTER, DROP AND TRUNCATE, are the four major sections of DDL.

Using CREATE command; we can create any database object including Tables, Views, Synonyms, Index.

MySQL allows Constraints to be defined at Table Level & at Column Level enforce the correct date entry from the end user.

CLASSIFICATION OF CONSTRAINTS:

In MySQL, Constraints can be classified either at the column level or at the Table level. Column level constraints will be defined immediately after the column names and their data types.

Table level constraints will be defined at the end after the column names are over.

Usually composite primary key or foreign keys are declared as Table constraints and rest all will be column constraints.

Naming of Constraints: In MySQL whenever we create a constraints either at the column level or at the Table level it compulsorily requires a Name, because the same name will be stored in the MySQL as Meta Data(information schema).

```
SELECT CONSTRAINT_NAME, CONSTRAINT_TYPE
FROM information_schema.TABLE_CONSTRAINTS
WHERE TABLE_NAME = 'EMP'
AND TABLE_SCHEMA = ‘CDAC’;
```

TABLE NAME SHOULD BE IN CAPITAL ONLY.

TYPES OF CONSTRAINTS: In MySQL, there are 5 different types of constraints. They are:-

(i). NOT NULL
(ii). UNIQUE
(iii). PRIMARY KEY
(iv). FOREIGN KEY
(v). CHECK

(i). **NOT NULL:** If implemented on one or more columns, this constraint will not allow Null or Blank values to that column. However, it does allow repetitive

values.

(ii). **UNIQUE:** When implemented on one or more column, this constraint does not allow repetitive values. However, it does allow NULL values.

(iii). **PRIMRY KEY:** It is a combination of NOT NULL and UNIQUE. It means when implemented on a particular Table, this column will neither allow NULL VALUE NOR ALLOW REPETITIVE VALUES. However, per table, ONLY ONE PRIMARY KEY IS ALLOWED.

However, if the situation demands, then we can create a composite primary key at the Table level by combining the columns. Maximum up to 16 columns we can combine.

(iv). **FOREIGN KEY:** When we want to link one Table with another by using a common column; then it has to be implemented by using rule FOREIGN KEY like COMPOSITE PRIMARY KEY. Usually, even FOREIGN KEY will be at the Table Level.

(v). **CHECK:** Whenever we want to Force certain special types of rules like ENAMES, entered must be CAPITAL ONLY job should be CLERK, MANAGER, SALESMAN ONLY or SLAARY should be less than 5000 only, and then it will be implemented by CHECK constraint.

REFERENCE: When we want to link one column with another within the same Table, like BOSSNO & EMPNO. Columns, then it has to be implemented by using the REFERENCE, but not FOREIGN KEY.

ON DELETE CASCADE: By default, whenever a child record exists, we will not be able to remove the Parent Key from the Parent Table. However, while creating the Foreign key, if we had given option ON DELETE CASCADE, then it allows us to remove the Parent Key even when Child records exists. But the moment, the primary Key is Deleted, All the corresponding Child records automatically get deleted.

This is a powerful (and slightly dangerous) tool! It's great for maintaining **Referential Integrity**.

- **Alternative:** You can also use `ON DELETE SET NULL`, which keeps the child record but sets the linking column to `NULL` instead of deleting the whole row.

DEFAULT: Even though officially not called, many people consider this as a constraint only. While creating the Table to a particular column, if we set certain DEFAULT VALUE If the END user omits its value in the INSERT statement, it will automatically use that DEFAULT value.

CREATING PARENT TABLE:

```
CREATE TABLE DPT (
   DEPTNO INT(2),
   DNAME VARCHAR(15),
   LOC VARCHAR(15) NOT NULL,

-- NOT NULL is an attribute, usually not named

   -- Naming the Primary Key
   CONSTRAINT PK_DEPT PRIMARY KEY (DEPTNO),

   -- Naming the Unique Constraint
   CONSTRAINT UQ_DEPT_NAME UNIQUE (DNAME)
);
```

CREATING CHILD TABLE:

```
CREATE TABLE AMP (
   EMPNO INT(4),
   ENAME VARCHAR(15) NOT NULL,
   JOB VARCHAR(15),
   BOSS_NO INT(4),
   DOJ DATE,
   SAL DECIMAL(7, 2),
   DEPTNO INT(2),

   -- Naming the Primary Key
   CONSTRAINT PK_EMP PRIMARY KEY (EMPNO),

   -- Naming the Self-Referencing Foreign Key (Boss link)
   CONSTRAINT FK_EMP_BOSS
      FOREIGN KEY (BOSS_NO) REFERENCES AMP (EMPNO),

   -- Naming the Department Foreign Key
   CONSTRAINT FK_EMP_DEPT
      FOREIGN KEY (DEPTNO) REFERENCES DPT (DEPTNO)
      ON DELETE CASCADE -- Added this to match your earlier notes!
);
```

<u>TABLE WITH COMPOSITE PRIMARY KEY</u>

```
CREATE TABLE customer (
   customer_id INT,
   account_id INT,
   customer_name VARCHAR(100) NOT NULL,
    CONSTRAINT pk_customer_account PRIMARY KEY(customer_id,
account_id));
```

TABLE WITH AUTO INCREMENT FEATURE

```
CREATE TABLE products (
   product_id INT AUTO_INCREMENT,
   product_name VARCHAR(100) NOT NULL,
   price DECIMAL(10, 2) NOT NULL,
   -- Setting the Primary Key
   CONSTRAINT pk_products PRIMARY KEY (product_id)
);
```

TO SEE METADATA IN MYSQL USE FOLLOWING COMMAND

```
SELECT
   TABLE_NAME AS 'View Name',
   TABLE_COMMENT AS 'Description'FROM
   INFORMATION_SCHEMA.TABLES
WHERE
   TABLE_SCHEMA = 'INFORMATION_SCHEMA';
```

TO SEE THE CONSTRAINTS APPLIED ON A TABLE TYPE FOLLOWING COMMAND

```
SELECT
   CONSTRAINT_NAME,
   CONSTRAINT_TYPE
FROM
   INFORMATION_SCHEMA.TABLE_CONSTRAINTS
WHERE
   TABLE_NAME = 'AMP'
   AND TABLE_SCHEMA = 'CDAC';
```

CREATION OF DUPLICATE TABLES:

CREATE TABLE EMP1 AS SELECT * FROM EMP;
Only NOT NULL constraint of the Original Table gets passed to the Duplicate Table also.

CREATION OF DUPLICATE STRUCTURE OF TABLES:

-- Step 1: Copy the structure EXACTLY (Keys, Indexes, etc.) CREATE TABLE EMP1 LIKE EMP;

THEN COPY DATA INTO TABLE:

-- Step 2: Copy the data INSERT INTO EMP1 SELECT * FROM EMP;

ALTER COMMAND

When We Want To Change The Structure of Table We Can Use DDL Command ALTER Which has Three Sub Options They Are
Add, Drop,Modify

Syntax: ALTER TABLE<NAME>[ADD,DROP,MODIFY]

Add Option is Used To Add a New Column or A New Constraint

Ex of Adding Column

Ex: ALTER TABLE AMP ADD Comm integer;

ALTER TABLE AMP ADD (Comm integer, Bonus integer);

ALTER TABLE EMP ADD COLUMN COMM INT AFTER ENAME;

ALTER TABLE EMP ADD COLUMN COMM INT FIRST;

-- If COMM already exists but is in the wrong place:

ALTER TABLE EMP MODIFY COLUMN COMM INT AFTER ENAME;

RENAME COLUMN

ALTER TABLE EMP RENAME COLUMN ENAME TO EMPNAME;

RENAME TABLE

ALTER TABLE EMP RENAME TO STAFF;

<u>Ex of Adding Constraint</u>

ALTER TABLE AMP ADD CONSTRAINT AMP_PKEY PRIMARY KEY (Empno);

ALTER TABLE AMP
ADD CONSTRAINT chk_ename_uppercase
CHECK (ename = UPPER(ename));

ALTER TABLE Amp ADD CONSTRAINT chk_min_salary CHECK (Sal >= 5000);

ALTER TABLE Amp ADD CONSTRAINT chk_job_title CHECK (Job IN ('CLERK', 'SALESMAN'));

Ex of Dropping Column

ALTER TABLE Amp DROP COLUMN Comm;

Ex of Dropping Constraint
ALTER TABLE Amp DROP FOREIGN KEY FK_EMP_BOSS;
Ex of Modify Column
ALTER TABLE Amp
MODIFY ename VARCHAR(15);

DROP COMMAND(DDL)

Drop Table STUDENT;

Drop Table DPT Cascade Constraints;(In Oracle)

Forcibly dropping parent Table When Child Table Exists

SET FOREIGN_KEY_CHECKS = 0;
DROP TABLE DPT;
SET FOREIGN_KEY_CHECKS = 1;

DIFFERENCE BETWEEN TRUNCATE & DELETE:

Truncate Command is equal to deleting all the records from the Table plus committing it. However, there are two major differences between DELTE AND TRUNCATE. In case of delete we can use ROLLBACK but in case of TRUNCATE, it is not possible. Rows will be deleted permanently.

Also in case of DELETE, rows will be removed only logically; means the space occupied by those rows will remain with the Table only; whereas in case of TRUNCATE, the rows will be removed permanently and the space occupied by those rows will be released to the Hard disk.

In Case of Delete Auto-Increment Keeps current And In case of Truncate count Resets count back to 1

TRUNCATE TABLE EMP1;

DATA CONTROL LANGUAGE:

Whenever we create a new user account by going to mysql -u root -p, then to give the permissions for doing the work in the new area, we use command GRANT and to take back the permissions, we use command REVOKE.

CREATE USER 'CDAC'@'localhost' IDENTIFIED BY 'BANGLORE';

GRANT CREATE ON *.* TO 'CDAC'@'localhost';

REVOKE CREATE ON *.* FROM 'CDAC'@'localhost'

DROP USER 'CDAC'@'localhost';

GRANT SELECT, INSERT, UPDATE, DELETE ON company_db.* TO 'CDAC'@'localhost';

OTHER DATABASE OBJECTS: VIEWS, SYNONYMS,INDEX

VIEWS:

1) A view is a virtual Table which does not exist, but appears to the end user as if it exists like a mirror image.

2) A view is an object through which partial data can be seen (or) changed.

3) A view is referred to a moon because it derives the data from the base table and presents us with the data.

4) A view can be created only by using SELECT statement.

5) On a view, we can do any DML operation, i.e., INSERT, UPDATE, DELETE exactly in the same way ado it on a Table; But whenever the operations are done on a view, it goes to the Table only because view is not having anything.

SECURITY:

Views are basically used as a part of Security, means; in many organizations, the end user (data entry operators) will never be give3n Original Tables & all the data entry will be done with the help of views only. But Data Base Administrator will be able to see everything because all the operations done by the different users will come to the same Table.

PROVIDING EXTRA INFORMATION:

Views can be used to provide extra information which does not exists even in the Original Table.

SUPPERESSING THE COMPLEX QUERIES:

Views are also used to suppress the compress/complicated queries like Sub queries/Correlated queries etc.,
Operations done on Views will be permanent not for temporary.

```
CREATE VIEW ABC AS SELECT EMPNO, ENAME,
(SAL + IFNULL (COMM, 0)) * 12 AS ANNUAL_INCOME FROM EMP;

CREATE VIEW ABC_JOIN AS
SELECT E. EMPNO, E. ENAME, D. DNAME, D.LOC
FROM EMP E, DEPT D
WHERE E. DEPTNO = D. DEPTNO;
```

CREATE VIEW D10 AS SELECT * FROM EMP WHERE DEPTNO = 10 WITH CHECK OPTION;

DROP VIEW D10;

SYNONYMS:

When we want to access the object belonging to another user, then we have to give the user name & object name every time we access the object. Instead of that, we can create a synonym also.

SELECT * FROM CDAC.EMP;
CREATE SYNONYM STAFF FOR CDAC.EMP;

In MySql -- Instead of CREATE SYNONYM STAFF FOR SCOTT.EMP;
CREATE VIEW STAFF AS SELECT * FROM CDAC.EMP;

INSERT INTO staff VALUES (...);

TEMPORARY TABLE

A **temporary table** in **MySQL** is a special table that exists only for the duration of a session (or until you explicitly drop it). It's mainly used to store **intermediate or short-term data** while working with queries.

Improve Performance (in some cases)

- If you need to reuse the same result multiple times, storing it once in a temp table can be faster than recalculating.

STEP 1

CREATE TEMPORARY TABLE temp_emp (empno INT, empname VARCHAR(50), job VARCHAR(50), sal DECIMAL(10, 2), deptno INT);

STEP 2

INSERT INTO temp_emp SELECT empno, ename, job, sal, deptno FROM emp WHERE sal >= 3000;

STEP 3

SELECT * FROM temp_emp;

INDEX:

MySQL Corporation strongly recommends the usage of INDEXES because it will help us in speeding up the query results. INDEXES are very useful when we are searching for a value in a repetitive date column like ENAME it is assumed that a index will speed up query results nearly by 30% especially on those Tables which have 1000 or more Number of rows.

Whenever we create an index, MySQL creates a separate internal data structure (typically a **B-Tree**) that stores the indexed column's values in a sorted order. Instead of a `ROWID`, each entry in this index contains the **Primary Key** value for the corresponding row. Our job is simply to define the index; the **MySQL Query Optimizer** automatically decides when using it is faster than scanning the whole table. When a query runs, MySQL searches the index to find the match, retrieves the associated Primary Key, and then uses that key to 'jump' directly to the full row in the main table—a process known as a **Clustered Key Lookup**."

```
-- Single Column Index
CREATE INDEX idx_ename ON EMP (ENAME);

-- Composite (Multiple Column) Index
CREATE INDEX idx_dept_name ON EMP (DEPTNO, ENAME);
```

Note: There is no need to create indexes on Primary key or Unique column because they are automatically indexed.